2D Heat/Advection Numerical Experiments

Amitaï, Yassine and Cédric

March 19, 2026

Agenda

PDEs Solved

Constant-coefficient diffusion

$$\frac{\partial u}{\partial t} = D\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right)$$

Variable-coefficient diffusion

$$\frac{\partial u}{\partial t} - \nabla \cdot (D(x,y)\nabla u) = 0$$

Variable-coefficient advection

$$\frac{\partial u}{\partial t} + \frac{\partial (v_x u)}{\partial x} + \frac{\partial (v_y u)}{\partial y} = 0$$

So far mostly ignored boundary conditions

Numerical Methods Used

Diffusion (explicit): Forward Euler + centered 5-point Laplacian

$$u_{i,j}^{n+1} = u_{i,j}^{n} + D\Delta t\left(\delta_{xx}u^{n} + \delta_{yy}u^{n}\right)$$

Diffusion (implicit): Backward Euler + sparse solve

$$(I - D\Delta t\,)u^{n+1} = u^{n}$$

Variable diffusion (implicit):

$$(I - \Delta t\, A(D))u^{n+1} = u^{n}$$

Advection upwind:

$$u_{i,j}^{n+1} = u_{i,j}^{n} - \Delta t\left(v_x\delta_x^{\text{up}}u + v_y\delta_y^{\text{up}}u\right), \quad \delta_x^{\text{up}} = \begin{cases} \dfrac{u_{i,j} - u_{i-1,j}}{\Delta x} & v_x > 0 \\ \dfrac{u_{i+1,j} - u_{i,j}}{\Delta x} & v_x < 0 \end{cases}$$

Advection MP5:

$$u^{n+1} = u^{n} - \frac{\Delta t}{\Delta x}\Delta_x F^{x} - \frac{\Delta t}{\Delta y}\Delta_y F^{y}$$

Numerical Methods Used

Diffusion (explicit): Forward Euler + centered 5-point Laplacian

$$u_{i,j}^{n+1} = u_{i,j}^{n} + D\Delta t\left(\delta_{xx}u^{n} + \delta_{yy}u^{n}\right)$$

Diffusion (implicit): Backward Euler + sparse solve

$$(I - D\Delta t\,)u^{n+1} = u^{n}$$

Variable diffusion (implicit):

$$(I - \Delta t\, A(D))u^{n+1} = u^{n}$$

Advection upwind:

$$u_{i,j}^{n+1} = u_{i,j}^{n} - \Delta t\left(v_x\delta_x^{\text{up}}u + v_y\delta_y^{\text{up}}u\right), \quad \delta_x^{\text{up}} = \begin{cases} \dfrac{u_{i,j} - u_{i-1,j}}{\Delta x} & v_x > 0 \\ \dfrac{u_{i+1,j} - u_{i,j}}{\Delta x} & v_x < 0 \end{cases}$$

Advection MP5:

$$u^{n+1} = u^{n} - \frac{\Delta t}{\Delta x}\Delta_x F^{x} - \frac{\Delta t}{\Delta y}\Delta_y F^{y}$$

Variable Diffusion - Implicit

Formula solved:

$$\partial_t u - \nabla \cdot (D(x, y)\nabla u) = 0$$

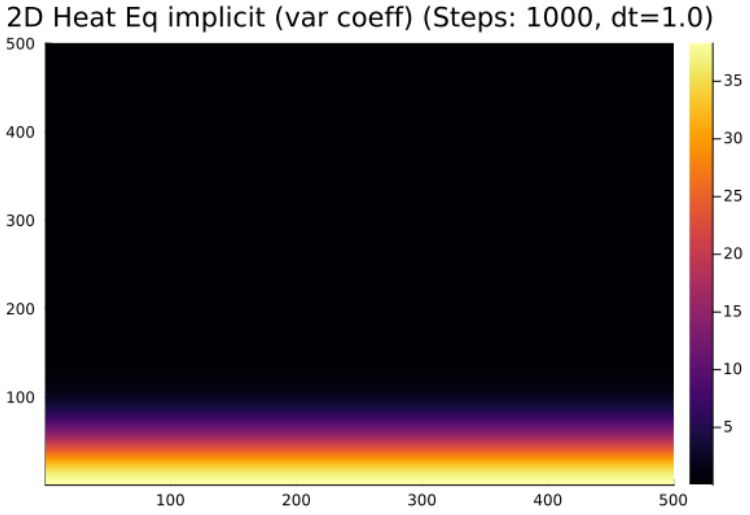
Method used: 2D Central Finite Differences, implicit Euler

Cases:

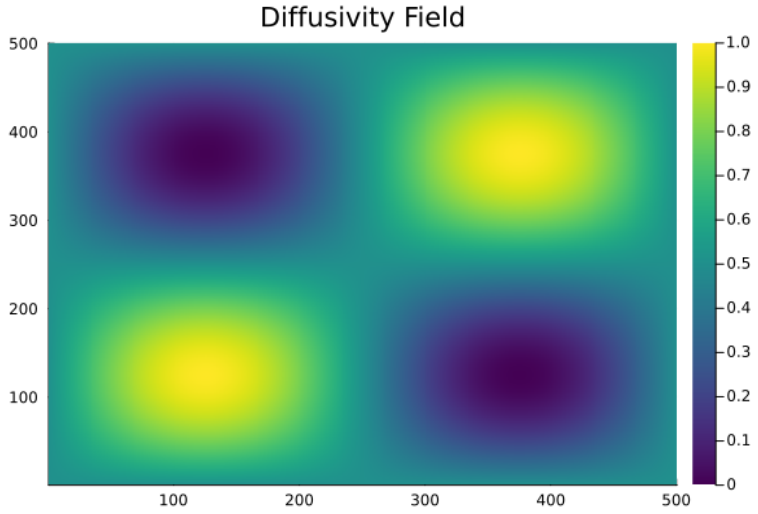
- Random $D(x, y)$, dt=1.0, start as box in middle
- Constant D, dt=1.0, start from lower wall
- Sinusoidal $D(x, y)$, dt=0.2, start as Gaussian blob in middle

Variable Diffusion Results - Random $D(x, y)$, dt=1.0, start as box in middle

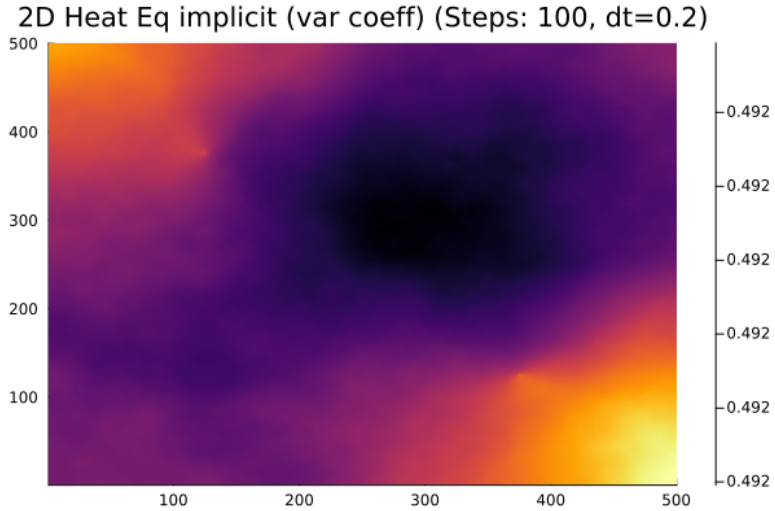
Variable Diffusion Results - Constant D, dt=1.0, start from lower wall



Sinusoidal $D(x, y)$, dt=0.2, start as Gaussian blob in middle



Sinusoidal $D(x, y)$, dt=0.2, start as Gaussian blob in middle



Variable Advection - Explicit Upwind

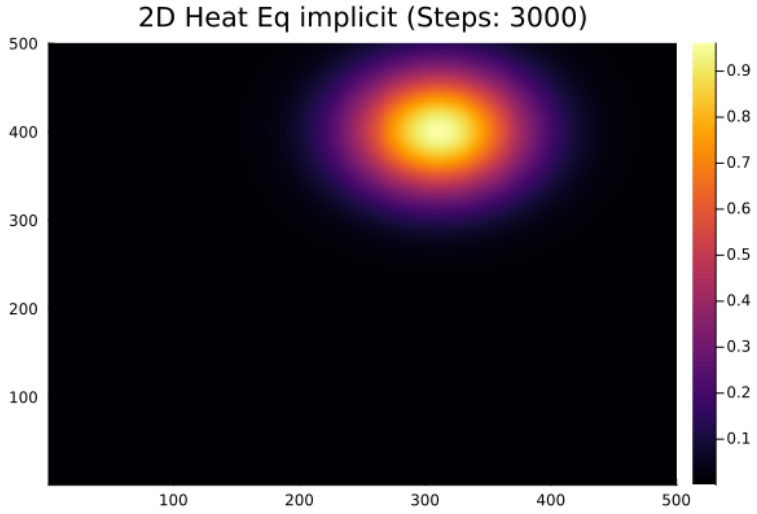
Formula solved:

$$\partial_t u + \partial_x(v_x u) + \partial_y(v_y u) = 0$$

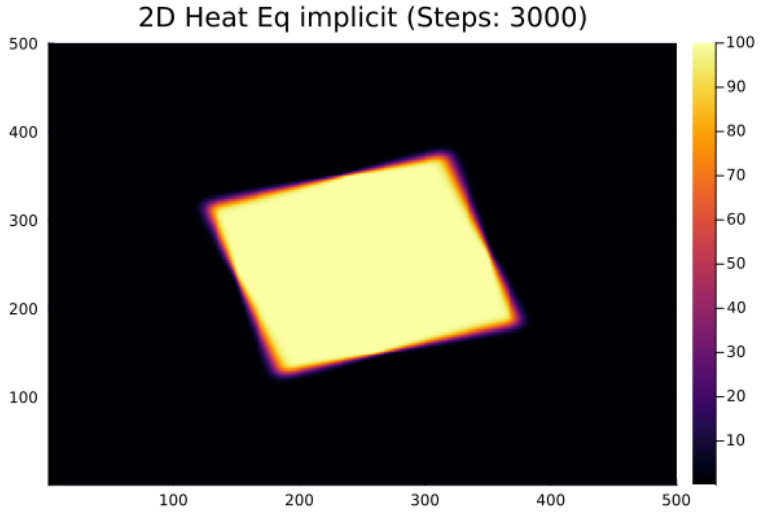
Method used: 2D first-order upwind + explicit Euler.
Cases:

- Constant velocity field $(v_x, v_y) = (0.5, 0.2)$ on Gaussian blob
- Rotational velocity field on square blob

Advection Upwind Results - Constant velocity field



Advection Upwind Results - Rotational velocity field



Variable Advection - MP5

Formula solved:

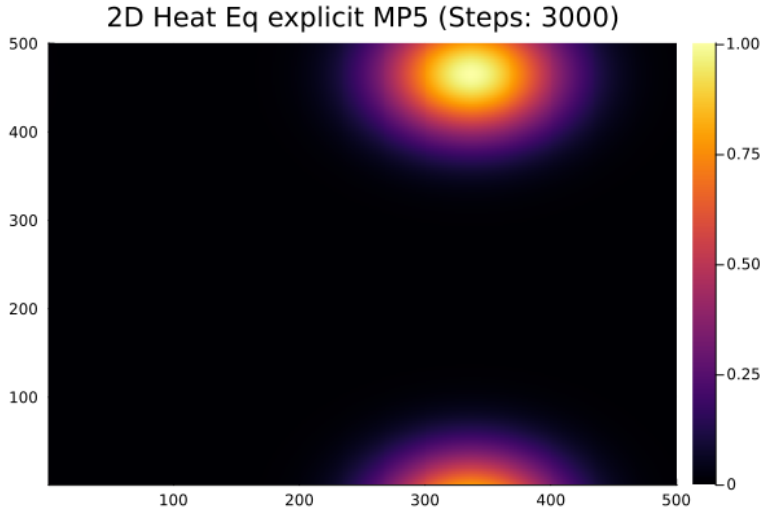
$$\partial_t u + \partial_x(v_x u) + \partial_y(v_y u) = 0$$

Method used: MP5 reconstruction in finite-volume fluxes + explicit Eueler (CFL-based Δt) (**BOTTLENECK!**).

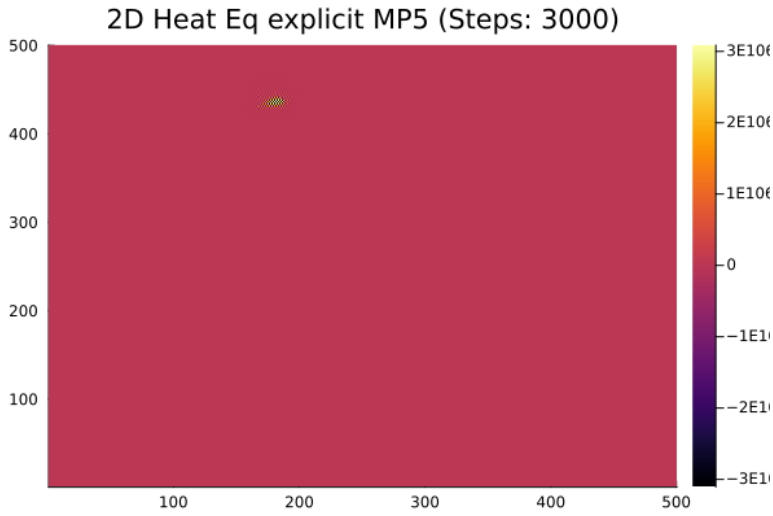
Cases:

- Constant velocity field $(v_x, v_y) = (0.5, 0.2)$ on Gaussian blob
- Rotational velocity field on square blob **troubles..**

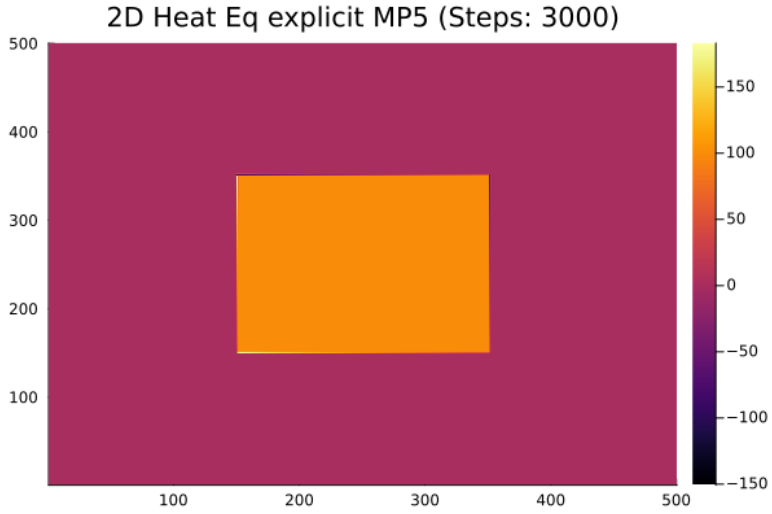
Advection MP5 Results - Constant velocity field



Advection MP5 Results - Rotational velocity field sqaure



Advection MP5 Results - Rotational velocity field sqaure



Manufactured Solution: Variable Diffusion + Source

Formula solved: (equidistant mesh)

$$\partial_t u - \nabla \cdot (D \nabla u) = S(x, y)$$

Method used: Implicit variable-diffusion + source term.
Manufactured fields:

$$u_{\text{exact}} = \cos(\pi x)\cos(\pi y), \quad D = 1 + x^2 + y^2$$

$S(x, y)$ gotten by inserting into PDE
Expected convergence: $\mathcal{O}(h^2)$ (spatial).

Manufactured Solution: Variable Diffusion + Source

Formula solved: (equidistant mesh)

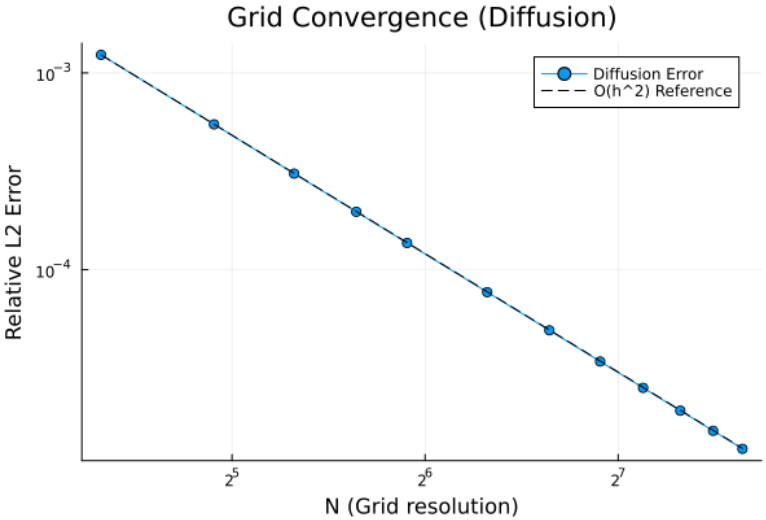
$$\partial_t u - \nabla \cdot (D \nabla u) = S(x, y)$$

Method used: Implicit variable-diffusion + source term.
Manufactured fields:

$$u_{\text{exact}} = \cos(\pi x)\cos(\pi y), \quad D = 1 + x^2 + y^2$$

$S(x, y)$ gotten by inserting into PDE
Expected convergence: $\mathcal{O}(h^2)$ (spatial).

Convergence Plot: Variable Diffusion



Manufactured Solution: Advection + Source

Formula solved:

$$\partial_t u + \partial_x(v_x u) + \partial_y(v_y u) = S(x, y)$$

Method used: First-order upwind explicit + source.

$$u_{\text{exact}} = \cos(\pi x)\cos(\pi y), \quad v_x = v_y = 1$$

Expected convergence: $\mathcal{O}(h)$.

Manufactured Solution: Advection + Source

Formula solved:

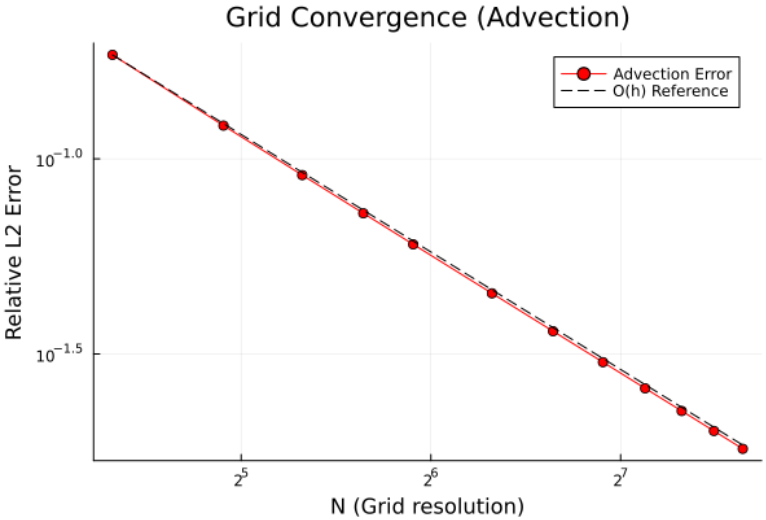
$$\partial_t u + \partial_x(v_x u) + \partial_y(v_y u) = S(x, y)$$

Method used: First-order upwind explicit + source.

$$u_{\text{exact}} = \cos(\pi x)\cos(\pi y), \quad v_x = v_y = 1$$

Expected convergence: $\mathcal{O}(h)$.

Convergence Plot: Advection Upwind



Advection Test: Rotating Gaussian (Variable Velocity)

Formula solved:

$$\partial_t u + \partial_x(v_x u) + \partial_y(v_y u) = 0$$

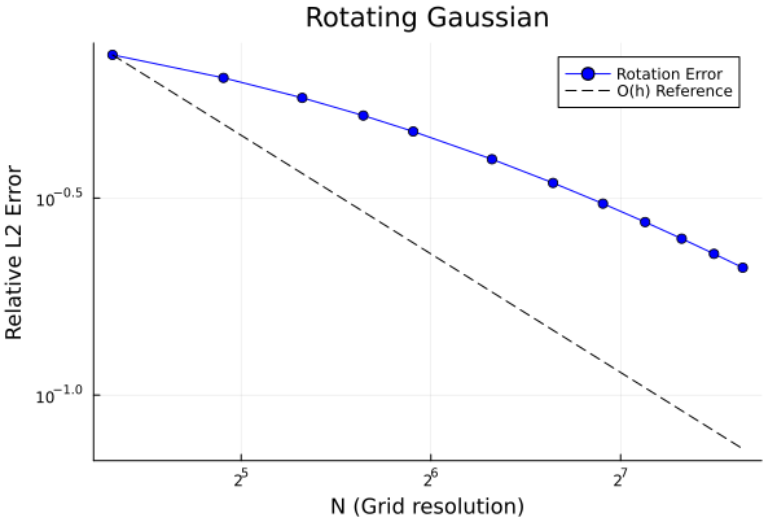
Method used: First-order upwind explicit (variable coefficients).

$$u_{\text{init}} = \exp\big(-\beta((x-x_0)^2 + (y-y_0)^2)\big)$$

$$v_x = -\omega(y-y_c), \quad v_y = \omega(x-x_c)$$

Exact solution: Rotation of the initial condition around (x_c, y_c).
Expected convergence: $\mathcal{O}(h)$.

Convergence Plot: Advection Upwind



Gaussian Blob Translation (MP5, No Source)

Formula solved:

$$\partial_t u + \partial_x(v_x u) + \partial_y(v_y u) = 0$$

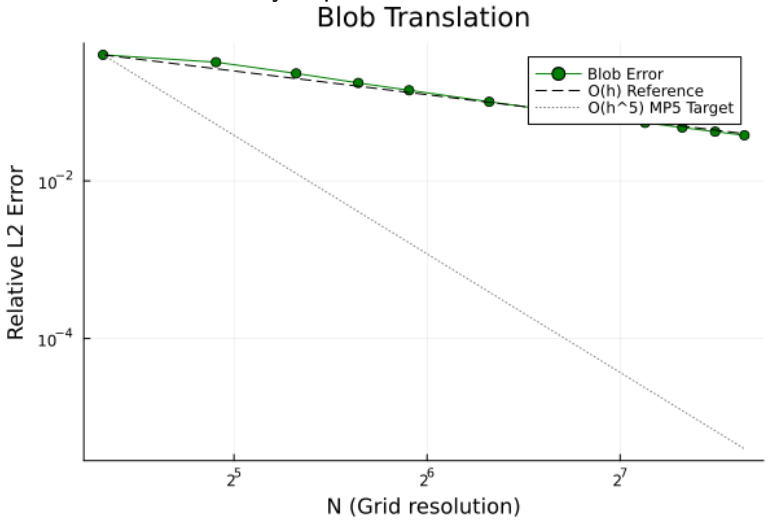
Method used: MP5 explicit advection.

$$u_{\text{exact}}(x, y, t) = \exp\left[-100\left((x - 0.2 - vt)^2 + (y - 0.2 - vt)^2\right)\right], \quad v_x = v_y = 1$$

Convergences: $\mathcal{O}(h)$ and $\mathcal{O}(h^5)$.

Convergence Plot: MP5 Blob Translation

dt calculation currently depends on $dx \Rightarrow$ still need to work on CFL condition



Discussion

Next steps:

- IMEX coding $\rightarrow$ use SplitODEProblem() from SciML
- Is our final goal IMEX or the time-splitting proposed in the 2020 paper?
- Rewrite the benchmarking stuff and plotting to be nicer (and more consistent)
- What was the thing about different speeds in the x and y directions to test? Already applicable?

Discussion:

Discussion

Next steps:

- IMEX coding → use SplitODEProblem() from SciML
- Is our final goal IMEX or the time-splitting proposed in the 2020 paper?
- Rewrite the benchmarking stuff and plotting to be nicer (and more consistent)
- What was the thing about different speeds in the x and y directions to test? Already applicable?

Discussion:

- We noticed that we have different background knowledge → Split up the project into a maths-heavy track and an implementation-heavy track to use skills more efficiently